

Components Data passing

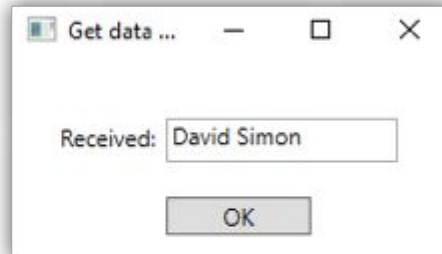
Main content

- ❑ Public static member
- ❑ Constructor parameter & Public property
- ❑ Delegate & event

Public static member

Example

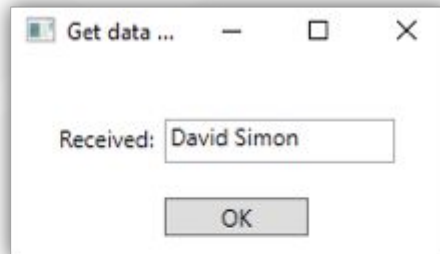
```
class Global
{
    public static string Data;
}
```



```
private void btnPassClick(object sender, RoutedEventArgs e)
{
    // Pass data to global
    Global.Data = txtName.Text;
    var screen = new GetGlobalWindow();
    screen.Show();
}
```

```
public GetGlobalWindow()
{
    InitializeComponent();
    // Get data from Global
    txtData.Text = Global.Data;
}
```

Thêm thành phần tĩnh cho màn hình đích



```
public static string Data;
```

```
private void btnPassClick(object sender, RoutedEventArgs e)
{
    // Store data before passing
    Data = txtName.Text;

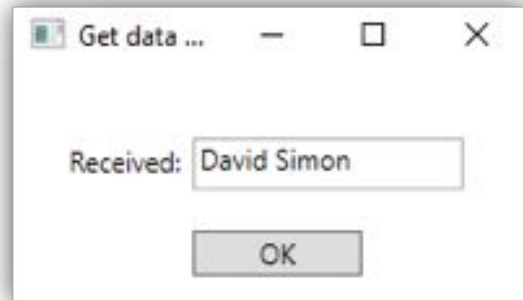
    var screen = new GetGlobalWindow();
    screen.Show();
}
```

```
public partial class GetGlobalWindow : Window
{
    public GetGlobalWindow()
    {
        InitializeComponent();

        // Get data source screen
        txtData.Text = MainWindow.Data;
    }
}
```

Constructor's parameter &
public property

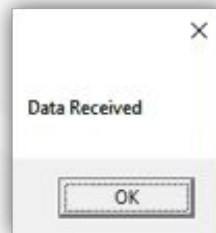
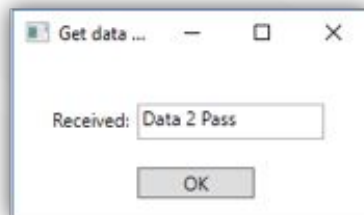
Using constructor to pass data



```
private void btnPassClick(object sender, RoutedEventArgs e)
{
    var screen = new GetGlobalWindow(txtName.Text);
    screen.Show();
}
```

```
public partial class GetGlobalWindow : Window
{
    public GetGlobalWindow(string data)
    {
        InitializeComponent();
        txtData.Text = data;
    }
}
```

Get data back using public property



```
public partial class MainWindow : Window
{
    public static string Data;

    private void btnPassClick(object sender, RoutedEventArgs e)
    {
        var screen = new GetGlobalWindow(txtName.Text);
        if (screen.ShowDialog().Value == true)
        {
            MessageBox.Show(screen.ReturnData);
        }
    }
}
```

```
public partial class GetGlobalWindow : Window
{
    public string ReturnData;

    public GetGlobalWindow(string data)
    {
        InitializeComponent();
        txtData.Text = data;
    }

    private void btnOK_Click(object sender, RoutedEventArgs e)
    {
        ReturnData = txtData.Text;
        DialogResult = true;
    }
}
```

Delegate & Event

What is delegate?

- ❑ **Type** of function
- ❑ Can be considered **data type**

```
delegate int Calculator(int a, int b);
```

Quiz: Which functions satisfy the Calculator type?

1. void checkCoexist(int a, int b)
2. int Multiply(int a, int b, int c)
3. int Power(int a, int b)
4. bool IsPrime(int a)

```
delegate int Calculator(int a, int b);
```

Basic examples

```
delegate int Calculator(int a, int b);

static int Sum(int a, int b)
{
    return a + b;
}

static int Multiply(int a, int b)
{
    return a * b;
}

static void Main(string[] args)
{
    Calculator func = Sum;
    Console.WriteLine(func(5, 7));

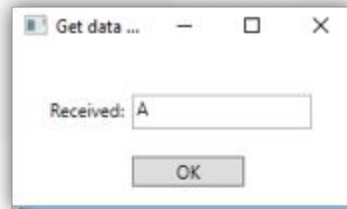
    func = Multiply;
    Console.WriteLine(func(3, 4));
}
```

What is event?

Array of pointer functions

```
delegate void ClickFunctionType(object sender, EventArgs e);  
event ClickFunctionType Click;  
  
public Form1()  
{  
    InitializeComponent();  
    Click += button1_Click;  
}  
  
private void button1_Click(object sender, EventArgs e)  
{  
}
```

Using delegate & event for receiving data in real time



```
private void btnPassClick(object sender, RoutedEventArgs e)
{
    var screen = new GetGlobalWindow(txtName.Text);
    screen.Handler += Screen_Handler;
    screen.ShowDialog();
}
private void Screen_Handler(string data)
{
    txtName.Text = data;
}
```

```
public delegate void MyDelegateType(string data);
public event MyDelegateType Handler;
```

```
private void txtData_TextChanged(object sender, TextChangedEventArgs e)
{
    if (Handler != null)
        Handler(txtData.Text);
}
```

Exercise

- ❑ Create a TODO app
- ❑ There will be a list of tasks that need to be done
- ❑ 1. Create a button to add a new task from another window
- ❑ 2. Edit a selected task in another window
- ❑ Try passing a `TodoTask` object rather than a string to learn more about reference

Problem - Restore back using backup?

- Step 1: Store a backup clone of current object
- Step 2: Update real time property on current object
 - Restore back if necessary
- Step 3: Copy all newly updated properties to current

Extension class for copy values - Warning: Shallow

```
public static class Reflection
{
    0 references
    public static void CopyProperties(this object source, object target)
    {
        Type sourceType = source.GetType();
        Type targetType = target.GetType();

        var sourceProps = source.GetType().GetProperties();

        foreach( var sourceProp in sourceProps )
        {
            if (! sourceProp.CanRead) continue;

            var targetProp = targetType.GetProperty( sourceProp.Name );
            if ( targetProp == null ) continue;
            if (! targetProp.CanWrite ) continue;
            if ((targetProp.GetSetMethod(true) != null)
                && targetProp.GetSetMethod(true)!.IsPrivate) continue;
            if ((targetProp.GetSetMethod()!.Attributes
                & System.Reflection.MethodAttributes.Static) != 0) continue;
            if (! targetProp.PropertyType.IsAssignableFrom( sourceProp.PropertyType ) ) continue;

            // Pass all tests, let's assign
            targetProp.SetValue(target, sourceProp.GetValue(source, null), null);
        }
    }
}
```

Full flow for preview - update - backup & restore

```
Employee _oldEmployee;  
1 reference  
private void updateButton_Click(object sender, RoutedEventArgs e)  
{  
    _oldEmployee = (Employee) _employee.Clone(); // backup  
  
    var screen = new EditEmployeeWindow(_employee);  
  
    screen.NewCreditsUpdating += Screen_NewCreditsUpdating;  
  
    if (screen.ShowDialog() == true)  
    {  
        Reflection.CopyProperties(screen.EditingEmployee, _employee);  
    } else // Restoring from the backup  
    {  
        Reflection.CopyProperties(_oldEmployee, _employee);  
    }  
}
```